DOTE 6635: Artificial Intelligence for Business Research (Spring 2026)

Deep Reinforcement Learning

Renyu (Philip) Zhang

1

Where Are We?

- Model-based methods
  - We know the action space, the state space, the transition probability matrix, and the reward function.
  - Bellman equation to evaluate a policy and Bellman optimality operator to find the optimal policy.
  - Value iteration, policy iteration, linear programming.
- Model-free methods
  - Monte Carlo (MC) learns directly from episodes of experience: Sample the entire reward process and use empirical mean return to approximate expected return.
  - Temporal Difference (TD) combines MC and Bellman operator: Bootstrap to update the value function based on the existing estimate.
  - MC has zero bias and high variance; TD has some bias and low variance.
- Model-free control
  - MC policy iteration combines MC policy evaluation and ε –greedy for policy improvement.
  - SARSA is an on-policy method to update Q(S,A) using TD and ε –greedy for policy improvement.
  - Q-learning is an off-policy method to learn Q(S,A) for the greedy target policy using ε –greedy as the behavior policy through TD update.

2

2

Agenda

- **Value Function Approximation**
- DQN
- Policy-based Methods

3

3

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- So far we assume that we can represent the MDP using state and action pairs (Q-function).
  - This is called tabular RL.
- What if the state or action spaces are too large?
  - Continuous states: self-driving car
  - Large states: Go has $3^{19*19} \approx 10^{137}$ states (# of atoms in universe: ~10^{80})
  - Continuous actions: prices
- We try to scale up the model-free methods using approximations for value/ policy functions:

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$

$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$$

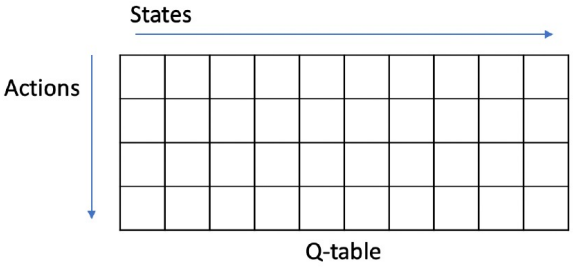
4

4

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In tabular RL, we represent the value functions by a lookup table:
  - Every state s has an entry $V(s)$
  - Every state-action pair (s,a) has an entry $Q(s,a)$



- Challenges with large MDPs:
  - Too many states or actions to store in memory
  - Too slow to learn the value of each state individually

5

5

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Avoid explicitly learning or storing for every single state
  - Dynamics and reward model
  - Value function $V(s)$ and state-action function $Q(s,a)$
  - Policy
- Solution: Estimate approximated value and policy functions:

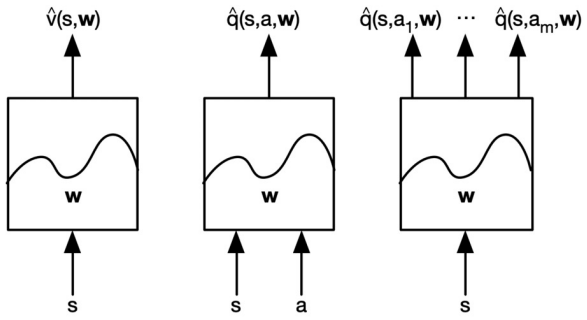
$$\hat{v}(s,\mathbf{w}) \approx v^{\pi}(s)$$

$$\hat{q}(s,a,\mathbf{w}) \approx q^{\pi}(s,a)$$

$$\hat{\pi}(a,s,\mathbf{w}) \approx \pi(a|s)$$

  - Generalize from seen states to unseen states
  - Update the parameter w using MC or TD learning.

Several Function Designs



6

6

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Consider differentiable functions as the approximators that can deal with non-stationary and non-iid data:
  - Linear feature representations
  - Neural Nets
- Idea: Use gradient descent to update the value or policy approximators every time we see a sample.
- Value function approximation with an oracle.
  - We have the oracle for knowing the trye value for $v^{\pi}(s)$ for any given state s.
  - The objective is to find the best approximate representation of $v^{\pi}(s)$.
  - Use the MSE with loss function as:

$$J(\mathbf{w}) = \mathbb{E}_{\pi}\left[\left(v^{\pi}(s) - \hat{v}(s, \mathbf{w})\right)^2\right]$$

  - Then gradient descent:

$$\Delta \mathbf{w} = -\frac{1}{2}\alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}$$

7

7

Linear Approximation with an Oracle

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Represent states using a finite vector with n variables: $\boldsymbol{x}(s) = \begin{pmatrix} x_1(s) \\ x_2(s) \\ \cdots \\ x_n(s) \end{pmatrix}$
- Represent value function by a linear combination of features: $\hat{V}(s; \boldsymbol{w}) = \sum_{j=1}^{n} x_j(s) w_j = \boldsymbol{x}(s)^T \boldsymbol{w}$
- Loss function: $J(\boldsymbol{w}) = \mathbb{E}_{\pi}[(V^{\pi}(s) - \hat{V}(s; \boldsymbol{w}))^2]$
- Stochastic gradient descent to update the weights: $\Delta \boldsymbol{w} = -\frac{1}{2}\alpha \nabla_{\boldsymbol{w}} J(\boldsymbol{w})$
  - Since the function is linear and the loss is MSE:

$$\Delta \mathbf{w} = \alpha(v^{\pi}(s) - \hat{v}(s, \mathbf{w}))\mathbf{x}(s)$$

  - The loss is convex in w, so SGD will converge to the global minimum.
  - With a general approximation function (SGD will only converge to the local minimum):

$$\Delta \mathbf{w} = \alpha\left(v^{\pi}(s) - \hat{v}(s, \mathbf{w})\right)\nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w})$$

8

8

Model Free Evaluation with Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In practice, we do not have oracles, just rewards collected by interacting with the environment.
- But we do obtain the target using samples.
- For MC, the target is simply the actual return G_t:

$$\Delta \mathbf{w} = \alpha\Big(G_t - \hat{v}(s_t, \mathbf{w})\Big)\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w})$$

  - G_t is an unbiased estimate of the oracle $v^{\pi}(s)$
  - MC prediction converges, in both linear and non-linear value function approximation.

- For TD(0), the target is the TD target $R_{t+1} + \gamma\hat{v}(s_{t+1}, \boldsymbol{w})$:

Called semi-gradient, as we ignore the effect of changing w on the target.

$$\Delta \mathbf{w} = \alpha\Big(R_{t+1} + \gamma\hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w})\Big)\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w})$$

  - TD target $R_{t+1} + \gamma\hat{v}(s_{t+1}, \boldsymbol{w})$ is a biased estimate of the oracle $v^{\pi}(s)$
  - Linear TD(0) converges (close) to global minimum.

9

9

Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

MC + SGD

while (not converged)

$s_0 \sim p_0,\ t = 0$
while ($s_t \neq$ <term>)
$a_t \sim \pi(\cdot \mid s_t)$
$(r_t, s_{t+1}) \sim p(\cdot, \cdot \mid s_t, a_t)$
$t = t + 1$
end
$T = t$

} sample trajectory τ

$$g = \left(V_\phi(s_0) - \sum_{t=0}^{T-1}\gamma^t r_t\right)\nabla_\phi V_\phi(s_0)$$

update ϕ using g with an optimizer

end

TD + (semi-)SGD

while (not converged)

$s_0 \sim p_0,\ a_0 \sim \pi(\cdot \mid s_0),\ (r_0, s_1) \sim p(\cdot, \cdot \mid s_0, a_0)$

$$y = \begin{cases} r_0 + \gamma V_\phi(s_1) & \text{if } s_1 \texttt{!=<term>} \\ r_0 & \text{if } s_1 \texttt{==<term>} \end{cases}$$

$g = (V_\phi(s_0) - y)\nabla_\phi V_\phi(s_0)$

update ϕ using g with an optimizer

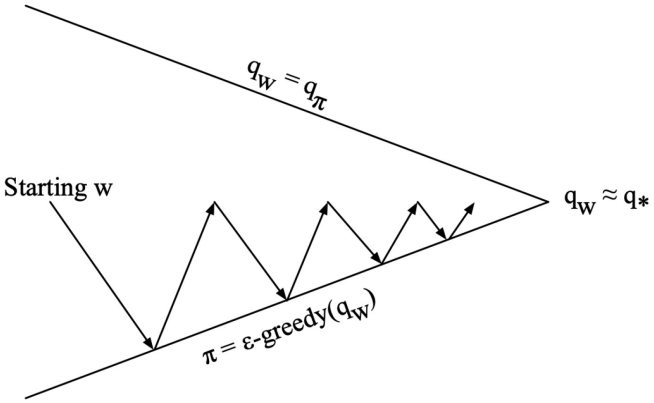
end

10

10

Generalized Policy Iteration with Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf



1. Policy evaluation: approximate policy evaluation, $\hat{q}(.,.,\mathbf{w}) \approx q^\pi$
2. Policy improvement: ϵ-greedy policy improvement

11

11

Model-free Control with Linear Approximation with an Oracle

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- With a similar idea, represent states using a finite vector with n variables: $\boldsymbol{x}(s,a) = \begin{pmatrix} x_1(s,a) \\ x_2(s,a) \\ \cdots \\ x_n(s,a) \end{pmatrix}$
- Use a linear combination of features to approximate Q-function so as to optimize over actions:

$$\hat{Q}(s,a;\boldsymbol{w}) = \boldsymbol{x}(s,a)^T \boldsymbol{w} = \sum_{j=1}^{n} x_j(s,a) w_j$$

- Minimize the MSE between the approximate Q-function and the oracle Q-function.

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))^2]$$

- Stochastic gradient descent update:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \nabla_{\boldsymbol{w}} \mathbb{E}_\pi[(Q^\pi(s,a) - \hat{Q}^\pi(s,a;\boldsymbol{w}))^2]$$

$$\Delta\mathbf{w} = \alpha(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(s,a,\mathbf{w}) \qquad \Delta\mathbf{w} = \alpha(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))\mathbf{x}(s,a)$$

12

12

Incremental Control Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In practice, we do not have oracle state-action function $q^{\pi}(s,a)$, just rewards collected by interacting with the environment.

- For MC, the target is simply the actual return G_t:

$$\Delta \mathbf{w} = \alpha\big(G_t - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

- For SARSA, the target is the TD target $R_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, \boldsymbol{w})$:

$$\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

- For Q-learning, the target is TD target $R_{t+1} + \gamma \max_a \gamma\hat{q}(s_{t+1}, a, \boldsymbol{w})$:

$$\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

13

13

Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

MC + SGD

while (not converged)

$s_0 \sim p_0,\ a_0 \sim \pi(\cdot \mid s_0),\ t = 0$
while ($s_t \neq$ <term>)
$a_t \sim \pi(\cdot \mid s_t)$
$(r_t, s_{t+1}) \sim p(\cdot, \cdot \mid s_t, a_t)$
$t = t + 1$
end
$T = t$

} sample trajectory τ

$$g = \Big(Q_\phi(s_0, a_0) - \sum_{t=0}^{T-1} \gamma^t r_t\Big)\nabla_\phi Q_\phi(s_0, a_0)$$

update ϕ using g with an optimizer

end

TD + (semi-)SGD

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
Repeat (for each episode):
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Repeat (for each step of episode):
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

14

14

Convergence of Control Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- SARSA: $\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$
- Q-learning: $\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$
- TD w. value function approximation does NOT follow the gradient of any loss function.
- The updates essentially involves approximate Bellman backup + fitting the underlying value function.
- TD may diverge in off-policy (e.g., Q-learning) or non-linear approximation settings.
- Challenge for off-policy control: Behavior policy and target policy are not identical, so value function approximation may diverge.

Convergence of Approximate Model-free Control

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	$\surd$	$(\surd)$	X
Sarsa	$\surd$	$(\surd)$	X
Q-Learning	$\surd$	X	X

$(\surd)$ moves around the near-optimal value function

Deadly Triad:
- Bootstrapping
- Function approximation
- Off-policy learning

15

15

Agenda

- Value Function Approximation
- **DQN**
- Policy-based Methods

16

16

Going Beyond Linear

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

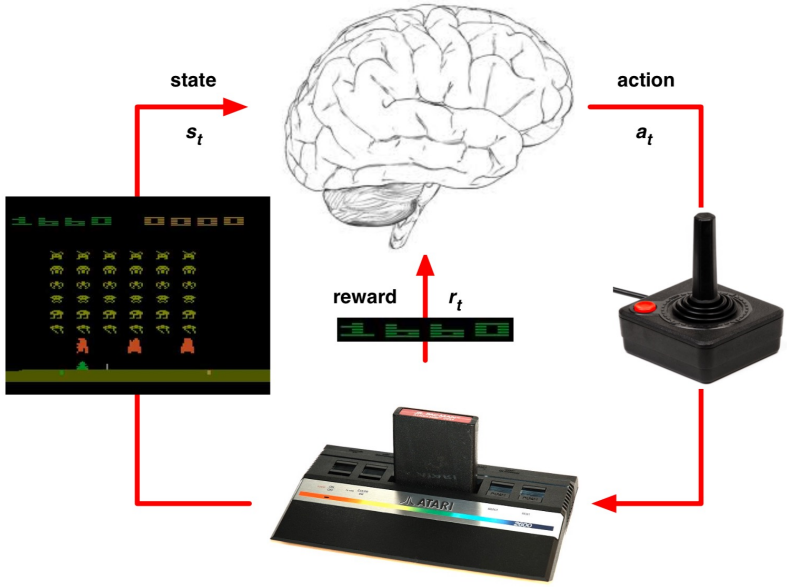
- So far, we have focused on linear approximators.
  - Linear approximations work well given the right set of features.
  - It requires manual design of the feature set.
- We know that DNN is a much better representation tool if we have a large data set.
- Question: How can we leverage deep learning for function approximation and model-free control of MDPs?

17

17

Deep Reinforcement Learning in Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf



18

18

Deep Q-Network (DQN) for Atari

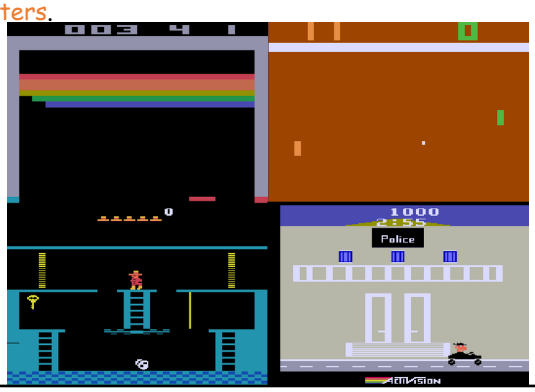
https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- Groud-breaking paper by DeepMind:

Human-level control through deep reinforcement learning
V Mnih, K Kavukcuoglu, D Silver, AA Rusu, J Veness… - nature, 2015 - nature.com
… large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner— illustrated by the temporal evolution of two indices of learning (the …
☆ 保存 99 引用 被引用次数：40192 相关文章 所有 55 个版本

- DQN represents the action-value function with DNN approximator.
- DQN reaches a professional human gaming level across many Atari games using the same network and hyperparameters.

Breakout



Pong

Montezuma's Revenge

Private Eye

Game	Linear	Deep Network
Breakout	3	3
Enduro	62	29
River Raid	2345	1453
Seaquest	656	275
Space Invaders	301	302

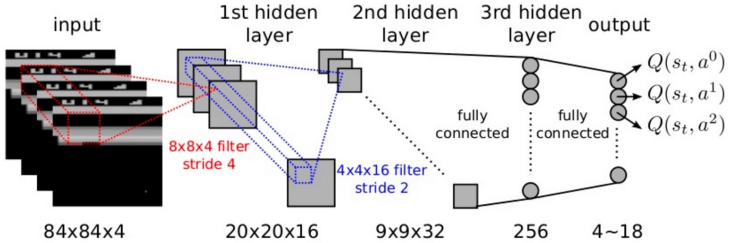
19

19

Deep Q-Network (DQN) for Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- End-to-end learning of $Q(s,a)$ from input fixel frame.
- Input state s is a stack of raw pixels from the latest 4 frames.
- Output of $Q(s,a)$ is 18 buttons.
- Reward is the change in score for that step.
- Network arthitecture and hyperparameters fixed across all games:



Abstract

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6,7,8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9,10,11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

20

20

Deep Reinforcement Learning

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- We leverage DNNs to represent:
  - Value functions (Q & V)
  - Policy functions (π)
  - World model
- Loss function is optimized by stochastic gradient descent (SGD)
- Core challenges:
  - Data inefficiency: Too many model parameters to optimize.
  - Deadly Triad creates instability and divergence in training:
    - Nonlinear function approximation
    - Bootstrapping
    - Off-policy training
- Deep Q-learning (DQN) addresses correlations between samples and non-stationary targets through:
  - Experience replay: Sample experience randomly from data to update weights, reducing correlations between data points.
  - Fixed Q-targets: Fix the target networks' weights while updating, improving stability.

21

21

DQN: Experience Replay

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- To reduce the correlations among samples, store the transition (s_t, a_t, r_t, s_{t+1}) in replay memory D.

s_1, a_1, r_2, s_2
s_2, a_2, r_3, s_3
s_3, a_3, r_4, s_4
...
$s_t, a_t, r_{t+1}, s_{t+1}$

$\rightarrow$ s, a, r, s'

- To perform experience replay, repeat the following:
  1. Sample an experience tuple from the dataset: $(s, a, r, s') \sim D$.
  2. Compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w})$
  3. Use SGD to update the network weights w:

$$\Delta \mathbf{w} = \alpha\left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}) - Q(s, a, \mathbf{w})\right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

- Use target as a scaler, but network weights will get updated on the next round, changing the target value.

22

22

DQN: Fixed Targets

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- To improve stability, fix the target weights used in the target calculation for multiple updates.
- Target network uses a different set of weights than the weights being updated.
- Let a different set of parameter $\mathbf{w}^-$, be the set of weights used in the target, and $\mathbf{w}$ be the weights that are being updated.
- To perform experience replay with fixed target, repeat the following:
  1. Sample an experience tuple from the dataset: $(s, a, r, s') \sim D$
  2. Compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-)$
  3. Use stochastic gradient descent to update the network weights:

$$\Delta \mathbf{w} = \alpha\left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-) - Q(s, a, \mathbf{w})\right)\nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

23

23

DQN Algorithm

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

1: Input C, α, $D = \{\}$, Initialize $\boldsymbol{w}$, $\boldsymbol{w}^- = \boldsymbol{w}$, $t = 0$
2: Get initial state s_0
3: **loop**
4: Sample action a_t given ϵ-greedy policy for current $\hat{Q}(s_t, a; \boldsymbol{w})$
5: Observe reward r_t and next state s_{t+1}
6: Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D
7: Sample random minibatch of tuples (s_i, a_i, r_i, s_{i+1}) from D
8: **for** j in minibatch **do**
9: **if** episode terminated at step $i + 1$ **then**
10: $y_i = r_i$
11: **else**
12: $y_i = r_i + \gamma \max_{a'} \hat{Q}(s_{i+1}, a'; \boldsymbol{w}^-)$
13: **end if**
14: Do gradient descent step on $(y_i - \hat{Q}(s_i, a_i; \boldsymbol{w}))^2$ for parameters $\boldsymbol{w}$: $\Delta \boldsymbol{w} = \alpha(y_i - \hat{Q}(s_i, a_i; \boldsymbol{w}))\nabla_{\boldsymbol{w}} \hat{Q}(s_i, a_i; \boldsymbol{w})$
15: **end for**
16: $t = t + 1$
17: **if** mod(t,C) == 0 **then**
18: $\boldsymbol{w}^- \leftarrow \boldsymbol{w}$
19: **end if**
20: **end loop**

Sample code:

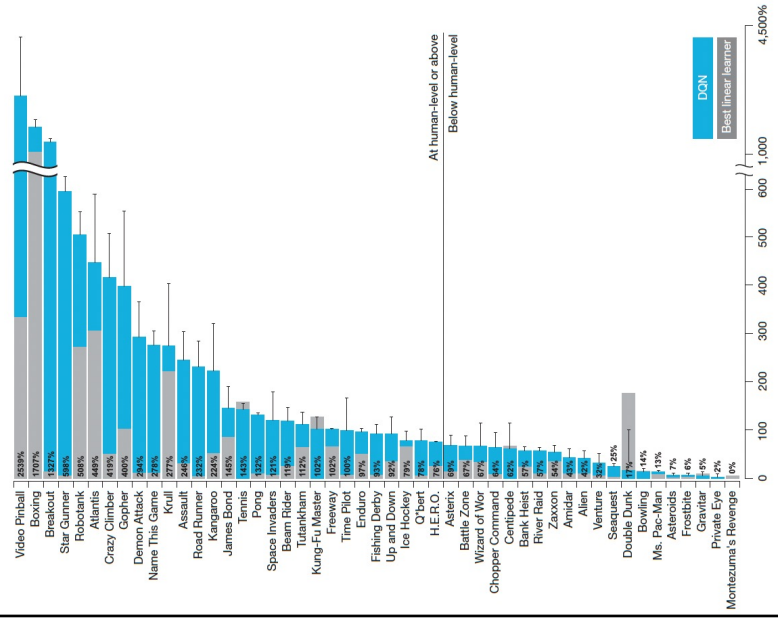
- https://www.tensorflow.org/agents/tutorials/0_intro_rl
- https://keras.io/examples/rl/deep_q_network_breakout/

24

24

Performance of DQN on Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf



25

25

Performance of DQN on Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

26

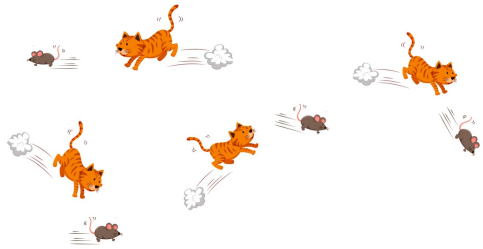
26

Why Fixed Targets?

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- In the original Q-learning with value function approximation, both Q estimation and Q target shifts at each time step.
- Think of a cat (Q estimation) is chasing a mouse (Q target), with the purpose that the cat reduces the distance to the mouse.

- If both the cat and the mouse are moving, we will observe an oscillated training history; so, we fix the target for a period of time during training.



27

27

Extensions to DQN

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- The success of DQN on Atari inspires a set of papers improving deep reinforcement learning.
  - Double DQN (AAAI 2016), Prioritized Replay (ICLR 2016), Dueling DQN (best paper at ICML 2016)
  - Also, huge economic values through self-driving cars.

Deep reinforcement learning with double q-learning
H Van Hasselt, A Guez, D Silver - … of the AAAI conference on artificial …, 2016 - ojs.aaai.org
… The popular Q-learning algorithm is known to overestimate … algorithm, which combines Q-learning with a deep neural network, … We then show that the idea behind the Double Q-learning …
☆ 保存 99 引用 被引用次数：12768 相关文章 所有 15 个版本 »

Prioritized experience replay
T Schaul, J Quan, I Antonoglou, D Silver - arXiv preprint arXiv:1511.05952, 2015 - arxiv.org
… To demonstrate the potential effectiveness of prioritizing replay by TD … prioritization' algorithm. This algorithm stores the last encountered TD error along with each transition in the replay …
☆ 保存 99 引用 被引用次数：7115 相关文章 所有 12 个版本 »

Dueling network architectures for deep reinforcement learning
Z Wang, T Schaul, M Hessel… - … machine learning, 2016 - proceedings.mlr.press
… Still, many of these applications use conventional architectures, such as convolutional … neural network architecture for model-free reinforcement learning. Our dueling network represents …
☆ 保存 99 引用 被引用次数：6631 相关文章 所有 8 个版本 »

28

28

Deep Q-Learning for Dynamic Coupon Targeting

informs
https://pubsonline.informs.org/journal/mksc

MARKETING SCIENCE
Articles in Advance, pp. 1–22
ISSN 0732-2399 (print), ISSN 1526-548X (online)

Dynamic Coupon Targeting Using Batch Deep Reinforcement Learning: An Application to Livestream Shopping

Xiao Liu[a]

[a]Stern School of Business, New York University, New York, New York 10012
Contact: xl23@stern.nyu.edu, https://orcid.org/0000-0002-7093-8534 (XL)

Received: September 3, 2020
Revised: September 5, 2021; April 17, 2022
Accepted: June 23, 2022
Published Online in Articles in Advance: October 20, 2022

https://doi.org/10.1287/mksc.2022.1403

Abstract. We present an empirical framework for creating dynamic coupon targeting strategies for high-dimensional and high-frequency settings, and we test its performance using a large-scale field experiment. The framework captures consumers' intertemporal tradeoffs associated with dynamic pricing and does not rely on functional form assumptions about consumers' decision-making processes. The model is estimated using batch deep reinforcement learning (BDRL), which relies on Q-learning, a model-free solution that can mitigate model bias. It leverages deep neural networks to represent the high-dimensional state space and alleviate the curse of dimensionality. The empirical application is in a multibillion-dollar livestream shopping context. Our BDRL solution increases the platform's revenue by twice as much as static targeting policies and by 20% more than the model-based solution. The comparative advantage of BDRL comes from more effective and automatic targeting of consumers based on both heterogeneity and dynamics, using exceptionally rich, nuanced differences among consumers and across time. We find that price skimming, reducing discounts for attractive hosts, and increasing the coupon discount level at a faster rate for low spenders are effective strategies based on dynamics, consumer heterogeneity, and the two combined, respectively.

History: K. Sudhir served as the senior editor and John Hauser served as associate editor for this article.
Funding: Partial financial support was received from the NYU Center for Global Economy and Business.
Supplemental Material: The data files and online appendices are available at https://doi.org/10.1287/mksc.2022.1403.

Keywords: dynamic pricing • coupon • deep reinforcement learning • reference price • livestream shopping • targeting

Dynamic coupon targeting using batch deep reinforcement learning: An application to livestream shopping
X Liu - Marketing Science, 2023 - pubsonline.informs.org
… incorporates the intertemporal tradeoffs in dynamic coupon targeting to design a policy … dynamic coupon targeting policies? (3) What are the gains, if any, from using a dynamic targeting …
☆ 保存 99 引用 被引用次数：115 相关文章 所有 7 个版本

Algorithm 2 (BCQ)

1. Input: Batch data $\mathcal{B}$, horizon T, target network update rate Υ, mini-batch size N, threshold τ.
2. Initialize Q-networks Q_θ, generative model G_ω (trained in a standard supervised learning fashion, with cross-entropy loss), and target networks $Q_{\theta'}$, with $\theta' \leftarrow \theta$.
3. for $t = 1$ to T, do:
4. Sample mini-batch M of N transitions $(\mathbf{S}, A, R, \mathbf{S}')$ from $\mathcal{B}$
5. $A' = \arg\max_{A' | \frac{G_\omega(A'|\mathbf{S}')}{\max_{\hat{A}} G_\omega(\hat{A}|\mathbf{S}')} > \tau} Q_\theta(\mathbf{S}', A')$
6. $\theta \leftarrow \arg\min_\theta \sum_{(\mathbf{S},A,R,\mathbf{S}') \in M} l_k(R + \gamma Q_{\theta'}(\mathbf{S}', A') - Q_\theta(\mathbf{S}, A))$
7. $\omega \leftarrow \arg\min_\omega - \sum_{(\mathbf{S},A) \in M} \log G_\omega(A \mid \mathbf{S})$
8. If $t \bmod \Upsilon = 0$: $\theta' \leftarrow \theta$
9. end for

Table 7. Model Comparison Based on the Doubly Robust Estimator

		1. Static policy with homogeneous treatments	2. Static policy with heterogeneous treatments			3. Model-based dynamic policy with heterogeneous treatments	4. Model-free dynamic policy with heterogeneous treatments
		A: Regression	B: GBDT	C: DNN	D: ORF	E: Structural[a]	F: Proposed BDRL
CLV (Return)	Mean	6.53	7.52	6.95	7.49	8.37	9.53
	Std	2.00	2.29	2.27	2.29	2.77	3.23
Gain	Mean	11%	28%	18%	28%	43%	62%
t test[b]		222.99	524.25	346.60	515.47	714.27	945.57
p value		<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

Notes. All the gains are compared with the mean CLV of ¥5.87. The total sample size is 1,020,898 consumers.

29

29

DRL for Ensembling Experimentation

informs
https://pubsonline.informs.org/journal/mnsc

MANAGEMENT SCIENCE
Vol. 70, No. 8, August 2024, pp. 5115–5130
ISSN 0025-1909 (print), ISSN 1526-5501 (online)

Ensemble Experiments to Optimize Interventions Along the Customer Journey: A Reinforcement Learning Approach

Yicheng Song,[a] **Tianshu Sun**[b,c,*]

[a]Carlson School of Management, University of Minnesota, Minneapolis, Minnesota 55455; [b]Center for Digital Transformation, Cheung Kong Graduate School of Business, Beijing 100006, China; [c]Marshall School of Business, University of Southern California, Los Angeles, California 90089
*Corresponding author
Contact: ycsong@umn.edu, https://orcid.org/0000-0002-9107-814X (YS); tianshusun@ckgsb.edu.cn, https://orcid.org/0000-0002-9786-044X (TS)

Received: December 6, 2021
Revised: July 6, 2022; November 22, 2022
Accepted: January 19, 2023
Published Online in Articles in Advance: October 4, 2023

https://doi.org/10.1287/mnsc.2023.4914

Abstract. Firms adopt randomized experiments to evaluate various interventions (e.g., website design, creative content, and pricing). However, most randomized experiments are designed to identify the impact of one specific intervention. The literature on randomized experiments lacks a holistic approach to optimize a sequence of interventions along the customer journey. Specifically, locally optimal interventions unveiled by randomized experiments might be globally suboptimal when considering their interdependence as well as the long-term rewards. Fortunately, the accumulation of a large number of historical experiments creates exogenous interventions at different stages along the customer journey and provides a new opportunity. This study integrates multiple experiments within the reinforcement learning (RL) framework to tackle the questions that cannot be answered by stand-alone randomized experiments. How can we learn optimal policy with a sequence of interventions along the customer journey based on an ensemble of historical experiments? Additionally, how can we learn from multiple historical experiments to guide future intervention trials? We propose a Bayesian recurrent Q-network model that leverages the exogenous interventions from multiple experiments to learn their effectiveness at different stages of the customer journey and optimize them for long-term rewards. Beyond optimization within the existing interventions, the Bayesian model also estimates the distribution of rewards, which can guide subject allocation in the design of future experiments to optimally balance exploration and exploitation. In summary, the proposed model creates a two-way complementarity between RL and randomized experiments, and thus, it provides a holistic approach to learning and optimizing interventions along the customer journey.

History: Accepted by Anindya Ghose, information systems.
Funding: This work was supported by Adobe Faculty Research Award and the Marketing Science Institute Research Grant.
Supplemental Material: The data files and online appendix are available at https://doi.org/10.1287/mnsc.2023.4914.

Keywords: reinforcement learning • customer journey • long-term reward optimization • Bayesian recurrent Q-network model (BRQN) • randomized experiment • experiment design

Ensemble experiments to optimize interventions along the customer journey: A reinforcement learning approach
Y Song, T Sun - Management Science, 2024 - pubsonline.informs.org
… of interventions along the customer journey based on an ensemble of historical experiments… we learn from multiple historical experiments to guide future intervention trials? We propose …
☆ 保存 99 引用 被引用次数：19 相关文章 所有 10 个版本

Algorithm 1 (Bayesian Recurrent *Q*-Network Estimation)

1: Set the empty reply buffer $RB()$, and load the historical experiment data into the buffer.
2: Initialize RNN parameter θ and Bayesian parameter w_a for each action.
3: Set N as the interval for Bayesian sampling and M as the interval for target model update.
4: Set $\theta_{target} = \theta$ and $w_{a,target} = w_a$.
5: **while** *step* <= num_{steps} **do**
6: Load a minibatch from $RB()$ with probability relative to TD error in Equation (8)
7: Get s_t based on *RNN*
8: Get s_{t+1} based on RNN_{target}
9: Get TD between state s_t and s_{t+1} based on Equation (8)
10: Update the RNN parameter θ based on Equation (9)
11: Bayesian posterior update based on Equations (3) and (4)
12: For every N steps: Thompson sampling $w_a \sim N(\mu, \Sigma)$
13: For every M steps: $\theta_{target} = \theta$ and $w_{a,target} = w_a$
14: *step* = *step* + 1
15: **end while**

Table 5. Average Reward per Episode (Cents) of BRQN with Different Experimental Data

	Fixed-M PERS @ 1	Fixed-M PERS @ 4	Fixed-M PERS @ 8
1 Experiment	502.4	2,027.9	4,190.2
5 Experiments	515.1	2,167.5	4,711.8
10 Experiments	536.5	2,429.3	5,708.6

Notes. We train the BRQN model with data from only 1 experiment (the one with the maximal number of treated users), 5 experiments (the top five with the greatest number of treated users), and all 10 experiments. Then, we adopt the fixed-M PERS to sample episodes with lengths of one, four, and eight. The average reward of the sampled episode across different settings shows promise of fueling RL with multiple experiments.

30

30

RL for Business Decision Making

informs
https://pubsonline.informs.org/journal/mnsc

MANAGEMENT SCIENCE
Vol. 69, No. 9, September 2023, pp. 5439–5460
ISSN 0025-1909 (print), ISSN 1526-5501 (online)

Deep Reinforcement Learning for Sequential Targeting

Wen Wang,[a] Beibei Li,[b,*] Xueming Luo,[c] Xiaoyi Wang[d]

[a] Robert H. Smith School of Business, University of Maryland, College Park, Maryland 20742; [b] Information Systems and Management, Carnegie Mellon University, Pittsburgh, Pennsylvania 15213; [c] Fox School of Business, Temple University, Philadelphia, Pennsylvania 19122; [d] School of Management, Zhejiang University, Hangzhou 310058, China

*Corresponding author

Contact: wenw@umd.edu, https://orcid.org/0000-0001-5983-3224 (WW); beibeili@andrew.cmu.edu, https://orcid.org/0000-0001-5466-7925 (BL); luoxm@temple.edu, https://orcid.org/0000-0002-5009-7854 (XL); kevinwxy@zju.edu.cn (XW)

Received: June 16, 2020
Revised: March 23, 2021; December 17, 2021
Accepted: April 18, 2022
Published Online in Articles in Advance: December 21, 2022

https://doi.org/10.1287/mnsc.2022.4621

Abstract. Deep reinforcement learning (DRL) has opened up many unprecedented opportunities in revolutionizing the digital marketing field. In this study, we designed a DRL-based personalized targeting strategy in a sequential setting. We show that the strategy is able to address three important challenges of sequential targeting: (1) forward looking (balancing between a firm's current revenue and future revenues), (2) earning while learning (maximizing profits while continuously learning through exploration-exploitation), and (3) scalability (coping with a high-dimensional state and policy space). We illustrate this through a novel design of a DRL-based artificial intelligence (AI) agent. To better adapt DRL to complex consumer behavior dimensions, we proposed a quantization-based uncertainty learning heuristic for efficient exploration-exploitation. Our policy evaluation results through simulation suggest that the proposed DRL agent generates 26.75% more long-term revenues than can the non-DRL approaches on average and learns 76.92% faster than the second fastest model among all benchmarks. Further, in order to better understand the potential underlying mechanisms, we conducted multiple interpretability analyses to explain the patterns of learned optimal policy at both the individual and population levels. Our findings provide important managerial-relevant and theory-consistent insights. For instance, consecutive price promotions at the beginning can capture price-sensitive consumers' immediate attention, whereas carefully spaced nonpromotional "cooldown" periods between price promotions can allow consumers to adjust their reference points. Additionally, consideration of future revenues is necessary from a long-term horizon, but weighing the future too much can also dampen revenues. In addition, analyses of heterogeneous treatment effects suggest that the optimal promotion sequence pattern highly varies across the consumer engagement stages. Overall, our study results demonstrate DRL's potential to optimize these strategies' combination to maximize long-term revenues.

History: Accepted by Kartik Hosanagar, information systems.
Supplemental Material: The data files and online appendix are available at https://doi.org/10.1287/mnsc.2022.4621.

Keywords: deep reinforcement learning • DRL • sequential targeting • promotions • forward looking • exploration-exploitation • scalability • AI

Deep reinforcement learning for sequential targeting
W Wang, B Li, X Luo, X Wang - Management Science, 2023 - pubsonline.informs.org
… based personalized targeting strategy in a sequential setting. … three important challenges of sequential targeting: (1) … while learning (maximizing profits while continuously learning …
☆ 保存 99 引用 被引用次数：53 相关文章 所有 8 个版本

informs
http://pubsonline.informs.org/journal/mnsc

MANAGEMENT SCIENCE
Vol. 67, No. 7, July 2021, pp. 4362–4383
ISSN 0025-1909 (print), ISSN 1526-5501 (online)

Demand-Aware Career Path Recommendations: A Reinforcement Learning Approach

Marios Kokkodis,[a] Panagiotis G. Ipeirotis[b]

[a] Carroll School of Management, Boston College, Chestnut Hill, Massachusetts 02467; [b] Leonard N. Stern School of Business, New York University, New York, New York 10012

Contact: kokkodis@bc.edu, https://orcid.org/0000-0002-5037-6060 (MK); panos@stern.nyu.edu, https://orcid.org/0000-0002-2966-7402 (PGI)

Received: October 8, 2017
Revised: February 6, 2019; January 3, 2020; May 6, 2020
Accepted: June 3, 2020
Published Online in Articles in Advance: October 8, 2020

https://doi.org/10.1287/mnsc.2020.3727

Abstract. A skill's value depends on dynamic market conditions. To remain marketable, contractors need to keep reskilling themselves continuously. But choosing new skills to learn is an inherently hard task: Contractors have very little information about current and future market conditions, which often results in poor learning choices. Recommendation frameworks could reduce uncertainty in learning choices. However, conventional approaches would likely be inefficient; they would model previous (often poor) observed contractor learning behaviors to provide future career path recommendations while ignoring current market trends. This work proposes a framework that combines reinforcement learning, Bayesian inference, and gradient boosting to provide recommendations on how contractors *should* behave when choosing new skills to learn. Compared with standard recommender systems, this framework does not learn from previous (often poor) behaviors to make future recommendations. Instead, it relies on a Markov decision process to operate on a graph of feasible actions and dynamically recommend profitable career paths. The framework uses market information to identify current trends and project future wages. Based on this information, it recommends feasible, relevant actions that a contractor can take to learn new, in-demand skills. Evaluation of the framework on 1.73 million job applications from an online labor market shows that its implementation could increase (1) the marketplace's revenue by up to 6%, (2) contractors' wages by 22%, and (3) the diversity of new skill acquisitions by 47%. A comparison with alternative recommender systems highlights the limitations of approaches that make recommendations based on previously observed learning behaviors.

History: Accepted by Chris Forman, information systems.
Supplemental Material: The online appendices are available at https://doi.org/10.1287/mnsc.2020.3727.

Keywords: skill recommendations • artificial intelligence • online labor markets • Markov decision process • information systems: enabling technologies

Demand-aware career path recommendations: A reinforcement learning approach
M Kokkodis, PG Ipeirotis - Management science, 2021 - pubsonline.informs.org
… Recommendation frameworks could reduce uncertainty in … to provide future career path recommendations while ignoring … boosting to provide recommendations on how contractors .
☆ 保存 99 引用 被引用次数：88 相关文章 所有 7 个版本 »»

31

31

Agenda

- Value Function Approximation
- DQN
- **Policy-based Methods**

32

32

From Value to Policy Approximations

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- So far, we have focused on approximating the value functions $\mathrm{v}^{\pi}(s)$ and $q^{\pi}(s,a)$ given the policy π.
- Another perspective is to parameterize the policy function $\pi_{\theta}(\mathrm{a}|\mathrm{s})$.
- Question: Solving the following policy optimization problem

$$\underset{\theta}{\text{maximize}} \quad \mathcal{J}(\theta) = \underset{s_0 \sim p_0}{\mathbb{E}} \left[V^{\pi_\theta}(s_0) \right] = \underset{s_0 \sim p_0}{\mathbb{E}^{\pi}} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]$$

where π_{θ} is represented by a DNN with parameter θ.

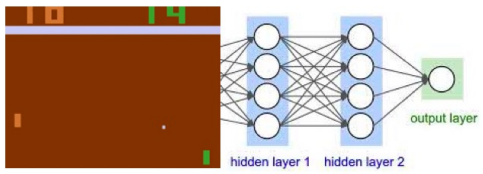
33

33

Policy Optimization

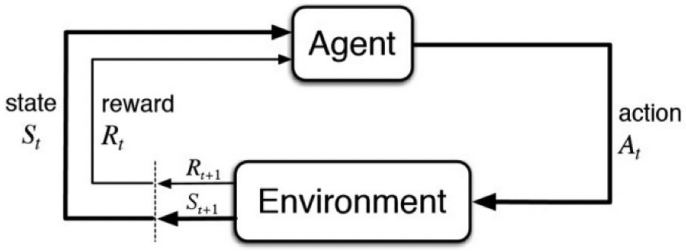
https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- The action and reward from the policy is all we need to optimize the policy directly.



Probability of moving up

$$\pi_{\theta}(A_t|S_t)$$



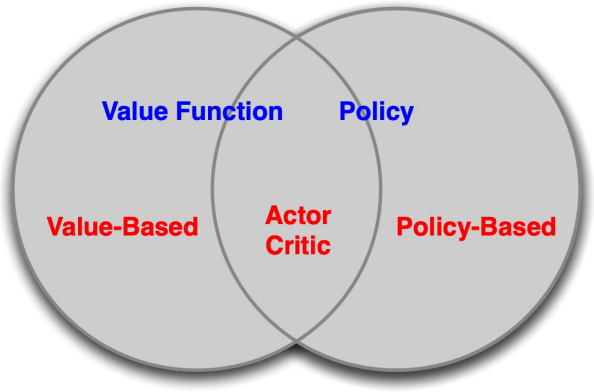
34

34

Value-based RL vs. Policy-based RL

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- Value-based RL learns the value function and optimizes the policy based on the learned value function.
- Policy-based RL learns the optimal policy directly without obtaining the value function.
- Actor-critic learns both policy and value functions.



35

35

Pros and Cons of Policy-based RL

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- We can directly learn stochastic policies (important for games).
- Policy-based RL is effective when action spaces are high-dimensional or continuous (e.g., LLMs).
- Better convergency properties, similar to policy iteration.
- Typically high variance.
- Typically convergence to local optimum.
  - The entire policy space is harder to optimize.

36

36

Policy Gradient for One-Step MDPs

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- Consider an MDP with one-step.
  - Start with $s \sim d(s)$
  - Terminate after one step with reward $r = R(s, a)$
- The reward of policy $\pi_\theta(s, a)$: $J(\theta) = \mathbb{E}_{\pi_\theta}[r]$

$$= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) r$$

- The gradient is:

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a) r$$

$$= \mathbb{E}_{\pi_\theta}[r \nabla_\theta \log \pi_\theta(s, a)]$$

$$\nabla_\theta \pi_\theta(s, a) = \pi_\theta(s, a) \frac{\nabla_\theta \pi_\theta(s, a)}{\pi_\theta(s, a)}$$

$$= \pi_\theta(s, a) \nabla_\theta \log \pi_\theta(s, a)$$

Score Function or Gradient of (Log-)Likelihood Ratio

37

37

Policy Gradient for Multi-Step MDPs

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- Consider a state-action trajectory from one episode as:

$$\tau = (s_0, a_0, r_1, \ldots s_{T-1}, a_{T-1}, r_T, s_T) \sim (\pi_\theta, P(s_{t+1} | s_t, a_t))$$

- Define $R(\tau) = \sum_{t=0}^{T} R(s_t, a_t)$ as the sum of rewards for a trajectory τ (assuming $\gamma = 1$).
- Policy value is:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} R(s_t, a_t)\right] = \sum_{\tau} P(\tau; \theta) R(\tau)$$

where $P(\tau; \theta) = \mu(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$ is the probability over the trajectory τ if policy π_θ is played.

- Our goal is to find the optimal policy parameter θ^*.

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta \sum_{\tau} P(\tau; \theta) R(\tau)$$

38

38

Taking Gradient

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta \sum_\tau P(\tau;\theta)R(\tau)$$

- Take the gradient w.r.t. θ:

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \sum_\tau P(\tau;\theta)R(\tau)\\ &= \sum_\tau \nabla_\theta P(\tau;\theta)R(\tau)\\ &= \sum_\tau \frac{P(\tau;\theta)}{P(\tau;\theta)}\nabla_\theta P(\tau;\theta)R(\tau)\\ &= \sum_\tau P(\tau;\theta)R(\tau)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}\\ &= \sum_\tau P(\tau;\theta)R(\tau)\nabla_\theta \log P(\tau;\theta)\end{aligned}$$

- Decompose $\nabla_\theta \log P(\tau;\theta)$:

$$\begin{aligned}\nabla_\theta \log P(\tau;\theta) &= \nabla_\theta \log\left[\mu(s_0)\prod_{t=0}^{T-1}\pi_\theta(a_t|s_t)p(s_{t+1}|s_t,a_t)\right]\\ &= \nabla_\theta\left[\log\mu(s_0) + \sum_{t=0}^{T-1}\log\pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t,a_t)\right]\\ &= \sum_{t=0}^{T-1}\nabla_\theta \log\pi_\theta(a_t|s_t)\end{aligned}$$

39

39

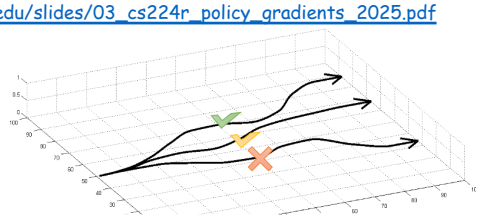
Estimate the Gradient and REINFORCE

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

$$\nabla_\theta J(\theta) = E_{\tau\sim p_\theta(\tau)}\left[\left(\sum_{t=1}^{T}\nabla_\theta \log\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)\right)\left(\sum_{t=1}^{T} r(\mathbf{s}_t,\mathbf{a}_t)\right)\right]$$

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta \log\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})\right)\left(\sum_{t=1}^{T} r(\mathbf{s}_{i,t},\mathbf{a}_{i,t})\right)$$

$$\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$$



Run policy to collect batch of data

Improve policy using batch of data

Full algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log\pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)\right)\left(\sum_t r(\mathbf{s}_t^i,\mathbf{a}_t^i)\right)$
3. $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$

"REINFORCE algorithm", vanilla policy gradient + Monte-Carlo

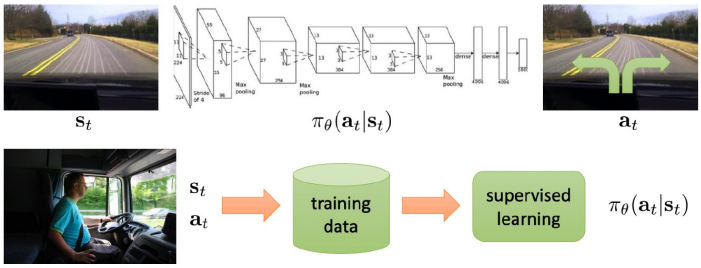
40

40

Policy Gradient vs. Maximum Likelihood

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

1. Policy gradient estimator: $\nabla_\theta J(\theta) \approx \frac{1}{M}\sum_{m=1}^{M}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{t,m}|s_{t,m})\right)\left(\sum_{t=1}^{T} r(s_{t,m}, a_{t,m})\right)$
2. Maximum likelihood estimator: (Also called imitation learning or behavioral cloning.) $\nabla_\theta J_{ML}(\theta) \approx \frac{1}{M}\sum_{m=1}^{M}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{t,m}|s_{t,m})\right)$
3. Interpretation: good action is made more likely, bad action is made less likely



41

41

Policy Gradient Theorem

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- The policy gradient theorem generalizes the likelihood ratio approach:

Theorem

For any differentiable policy $\pi_\theta(s,a)$, for any of the policy objective function $J = J_1$, (episodic reward), J_{avR} (average reward per time step), or $\frac{1}{1-\gamma}J_{avV}$ (average value), the policy gradient is

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s,a) Q^{\pi_\theta}(s,a)]$$

- Basically, the policy gradient theorem says policy gradient = Q-function following the policy weighted by the score function of the policy.

42

42

Score Function

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf

- Discrete action policy: policy function is softmax, and the score function is as follows:

$$\pi_\theta(s,a) = \frac{\exp^{\phi(s,a)^T\theta}}{\sum_{a'}\exp^{\phi(s,a')^T\theta}} \qquad \nabla_\theta \log \pi_\theta(s,a) = \phi(s,a) - \mathbb{E}_{\pi_\theta}[\phi(s,.)]$$

- Continuous action policy: policy function is Gaussian with mean linear in states, and the score function is as follows:

$$\mu(s) = \phi(s)^T\theta$$
$$a \sim \mathcal{N}(\mu(s), \sigma^2)$$

$$\nabla_\theta \log \pi_\theta(s,a) = \frac{(a-\mu(s))\phi(s)}{\sigma^2}$$

43

43

Large Variance of Policy Gradient

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

- We have the following approximate gradient estimate:

$$\nabla_\theta J(\theta) \approx \frac{1}{m}\sum_{i=1}^{m} R(\tau_i)\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t^i|s_t^i)$$

- This is unbiased but very noisy.

- Two fixes:
  - Leverage temporal causality.
  - Subtract a baseline.

44

44

Temporal Causality

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

- Temporal causality: policy at time t' cannot affect reward at time t for $t < t'$

$$\nabla_\theta \mathbb{E}_\tau[r_{t'}] = \mathbb{E}_\tau\left[r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

$$\begin{aligned}\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R] &= \mathbb{E}_\tau\left[\sum_{t'=0}^{T-1} r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t)\right] \\ &= \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{t'=t}^{T-1} r_{t'}\right] \\ &= \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)\right]\end{aligned}$$

We have the following estimated gradient update: $\nabla_\theta \mathbb{E}[R] \approx \frac{1}{m} \sum_{i=1}^{m} \sum_{t=0}^{T-1} G_t^{(i)} \cdot \nabla_\theta \log \pi_\theta(a_t^i|s_t^i)$

45

Subtracting Baseline

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R] = \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

- $G_t = \sum_{t'=t}^{T-1} r_{t'}$ is the return for a trajectory which might have high variance.
- We can subtract a constant baseline $b(s_t) = \mathbb{E}[r_t + r_{t+1} + \ldots + r_{T-1}]$

$$\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R] = \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} (G_t - b(s_t)) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

Increase the log-prob of the action proportional to how much returns are better than expectation.

$$\mathbb{E}_\tau\left[\nabla_\theta \log \pi_\theta(a_t|s_t) b(s_t)\right] = 0,$$

$$E_\tau\left[\nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t))\right] = E_\tau\left[\nabla_\theta \log \pi_\theta(a_t|s_t) G_t\right]$$

$$Var_\tau\left[\nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t))\right] < Var_\tau\left[\nabla_\theta \log \pi_\theta(a_t|s_t) G_t\right]$$

Subtracting the baseline is unbiased and reduces variance.

46

Vanilla Policy Gradient with Baseline

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

procedure POLICY GRADIENT(α)
Initialize policy parameters θ and baseline values $b(s)$ for all s, e.g. to 0
for iteration $= 1, 2, \ldots$ **do**
Collect a set of m trajectories by executing the current policy π_θ
for each time step t of each trajectory $\tau^{(i)}$ **do**
Compute the *return* $G_t^{(i)} = \sum_{t'=t}^{T-1} r_{t'}$
Compute the *advantage estimate* $\hat{A}_t^{(i)} = G_t^{(i)} - b(s_t)$
Re-fit the baseline to the empirical returns by updating $\mathbf{w}$ to minimize

$$\sum_{i=1}^{m} \sum_{t=0}^{T-1} \|b(s_t) - G_t^{(i)}\|^2$$

Update policy parameters θ using the policy gradient estimate $\hat{g}$

$$\hat{g} = \sum_{i=1}^{m} \sum_{t=0}^{T-1} \hat{A}_t^{(i)} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$$

with an optimizer like SGD ($\theta \leftarrow \theta + \alpha \cdot \hat{g}$) or Adam
return θ and baseline values $b(s)$

- Vanilla policy gradient is on-policy, i.e., update uses only data from the current policy.
- How to generalize policy gradient to off-policy, i.e., update can reuse data from other, past policies?

47

47

Off-policy Version of Policy Gradient

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

- What if we want to use samples collected from a different policy, π?

$$J(\theta) = E_{\tau \sim p_\theta(\tau)}[r(\tau)]$$

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)}\left[\frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau)\right]$$

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{p(\mathbf{s}_1)\prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)}{p(\mathbf{s}_1)\prod_{t=1}^{T} \bar{\pi}(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} = \frac{\prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\prod_{t=1}^{T} \bar{\pi}(\mathbf{a}_t|\mathbf{s}_t)}$$

Say we want to update our latest policy $\pi_{\theta'}$ but we want to use samples from π_θ

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)}\left[\left(\sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)\right)\left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})\right) - b\right)\right]$$

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)}\left[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)}\left(\sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)\right)\left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})\right) - b\right)\right]$$

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)}\left[\prod_{t=1}^{T} \frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left(\sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)\right)\left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})\right) - b\right)\right]$$

This can become very small or very large, for larger T

Importance sampling

Using proposal distribution q

$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)}\left[\frac{p(x)}{q(x)} f(x)\right] \end{aligned}$$

Note: Important for q to have non-zero support for high probability $p(x)$

48

48

Off-policy Version of Policy Gradient

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

Say we want to update our latest policy $\pi_{\theta'}$ but we want to use samples from π_θ

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\prod_{t=1}^{T} \frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \left(\sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \right) \left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right) - b \right) \right]$$

This can become very small or very large, for larger T

What if we consider the expectation *over timesteps* instead of trajectories?

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \frac{\pi_{\theta'}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})}{\pi_\theta(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

Much less likely to explode/vanish ...but, hard to measure $\frac{\pi_{\theta'}(\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{s}_{i,t})}$ *often approximated as 1*

Common final form

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^{T} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

Full algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
2. compute $\nabla_\theta J(\theta)$ using $\{\tau^i\}$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Can take multiple gradient steps on the same batch

One common choice: $\mathbb{E}_{\mathbf{s} \sim \pi_\theta} [D_{KL}(\pi_{\theta'}(\cdot \mid \mathbf{s}) \| \pi_\theta(\cdot \mid \mathbf{s}))] \leq \delta$

- If a policy changes too much before sampling new data, the data no longer reflects states the policy will visit.
- Need to constrain the policy not to change too much.

49

Reducing Variance with a Critic

https://web.stanford.edu/class/cs234/slides/lecture5pre.pdf; https://cs224r.stanford.edu/slides/03_cs224r_policy_gradients_2025.pdf

- Recall the policy gradient update:

$$\nabla_\theta \mathbb{E}[R] \approx (1/m) \sum_{i=1}^{m} \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t, s_t)(G_t^{(i)} - b(s_t))$$

- The MC rollout return G_t^i is an unbiased estimate of the Q-function, but with a high variance.
- We can instead use a critic to better estimate the Q-function to reduce its variance.
  - Critic: Updates Q-function with a parameter w, which is policy evaluation through approximation.
  - Actor: Updates policy function using the Q-function produced by critic.
  - Baseline: Use the average reward $V(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\cdot|\mathbf{s}_t)}[Q(\mathbf{s}_t, \mathbf{a}_t)]$
- We only need to estimate the difference between Q-function and the baseline, called the advantage function, which can be approximated by the TD-error.

$$A^{\pi_\theta}(s,a) = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s,a)\ A^{\pi_\theta}(s,a)]$$

$$\delta^{\pi_\theta} = r + \gamma V^{\pi_\theta}(s') - V^{\pi_\theta}(s)$$

$$\begin{aligned} \mathbb{E}_{\pi_\theta}[\delta^{\pi_\theta}|s,a] &= \mathbb{E}_{\pi_\theta}[r + \gamma V^{\pi_\theta}(s')|s,a] - V^{\pi_\theta}(s) \\ &= Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) \\ &= A^{\pi_\theta}(s,a) \end{aligned}$$

50

Compatible Function Approximation Theorem

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-7-policy-gradient-methods.pdf

Theorem (Compatible Function Approximation Theorem)

If the following two conditions are satisfied:

1. *Value function approximator is compatible to the policy*

$$\nabla_w Q_w(s,a) = \nabla_\theta \log \pi_\theta(s,a)$$

2. *Value function parameters w minimise the mean-squared error*

$$\varepsilon = \mathbb{E}_{\pi_\theta}\left[(Q^{\pi_\theta}(s,a) - Q_w(s,a))^2\right]$$

Then the policy gradient is exact,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ Q_w(s,a)\right]$$

- Q-function approximation has errors. Does it affect the policy gradient updates?
- This theorem says that not really if the error satisfies some properties.

51

51

Simple Action-Value Actor-Critic Algorithm

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-7-policy-gradient-methods.pdf

- Use a linear value function approximation: $Q_{\mathbf{w}}(s,a) = \psi(s,a)^T \mathbf{w}$
  - Critic: Update w by a linear TD(0).
  - Actor: Update θ by policy gradient.

Algorithm 2 Simple QAC

1: **for** each step **do**
2: generate sample s, a, r, s', a' following π_θ
3: $\delta = r + \gamma Q_{\mathbf{w}}(s', a') - Q_{\mathbf{w}}(s,a)$ #TD error
4: $\mathbf{w} \leftarrow \mathbf{w} + \beta\delta\psi(s,a)$
5: $\theta \leftarrow \theta + \alpha\nabla_\theta \log \pi_\theta(s,a) Q_{\mathbf{w}}(s,a)$
6: **end for**

52

52

A3C Algorithm

https://lilianweng.github.io/posts/2018-04-08-policy-gradient/

Asynchronous methods for deep reinforcement learning
V Mnih, AP Badia, M Mirza, A Graves… - … machine learning, 2016 - proceedings.mlr.press
… DQN was trained on a single Nvidia K40 GPU while the asynchronous methods were …
asynchronous methods we average over the best 5 models from 50 experiments with learning …
☆ 保存 99 引用 被引用次数：14525 相关文章 所有 22 个版本 »

- Asynchronous advantage actor-critic (A3C) is a policy gradient method with parallel training (of actors).
  - Critics learn the value function while multiple actors are trained in parallel.

1. We have global parameters, θ and w; similar thread-specific parameters, θ' and w'.
2. Initialize the time step $t = 1$
3. While $T \leq T_{\text{MAX}}$:
   1. Reset gradient: $\mathrm{d}\theta = 0$ and $\mathrm{d}w = 0$.
   2. Synchronize thread-specific parameters with global ones: $\theta' = \theta$ and $w' = w$.
   3. t_{start} = t and sample a starting state s_t.
   4. While (s_t != TERMINAL) and $t - t_{\text{start}} \leq t_{\max}$:
      1. Pick the action $A_t \sim \pi_{\theta'}(A_t|S_t)$ and receive a new reward R_t and a new state s_{t+1}.
      2. Update $t = t + 1$ and $T = T + 1$
   5. Initialize the variable that holds the return estimation

$$R = \begin{cases} 0 & \text{if } s_t \text{ is TERMINAL} \\ V_{w'}(s_t) & \text{otherwise} \end{cases}$$

6. For $i = t-1, \ldots, t\_start$: 1. $R \leftarrow \gamma R + R\_i$; here R is a MC measure of G_i. 2. Accumulate gradients w.r.t. θ': $d\theta \leftarrow d\theta + \nabla\_\theta' \log \pi\_\theta'(a\_i|s\_i)(R - V\_w'(s\_i))$; Accumulate gradients w.r.t. w': $dw \leftarrow dw + 2(R - V\_w'(s\_i))\nabla\_w'(R - V\_w'(s\_i))$.
7. Update asynchronously θ using $\mathrm{d}\theta$, and w using $\mathrm{d}w$.

https://github.com/dennybritz/reinforcement-learning/tree/master/PolicyGradient/a3c

53

53

DRL for Inventory Control

informs
http://pubsonline.informs.org/journal/msom
MANUFACTURING & SERVICE OPERATIONS MANAGEMENT
Vol. 24, No. 3, May–June 2022, pp. 1349–1368
ISSN 1523-4614 (print), ISSN 1526-5498 (online)

Can Deep Reinforcement Learning Improve Inventory Management? Performance on Lost Sales, Dual-Sourcing, and Multi-Echelon Problems

Joren Gijsbrechts,[a,*] Robert N. Boute,[b,c] Jan A. Van Mieghem,[d] Dennis J. Zhang[e]
[a] Universidade Católica Portuguesa, Católica Lisbon School of Business and Economics, 1649-023 Lisbon, Portugal; [b] Vlerick Business School, Technology and Operations Management Area, 3000 Leuven, Belgium; [c] Katholieke Universiteit Leuven, Research Center for Operations Management, 3000 Leuven, Belgium; [d] Kellogg School of Management, Northwestern University, Evanston, Illinois 60208; [e] Olin Business School, Washington University in St. Louis, St. Louis, Missouri 63130
*Corresponding author
Contact: jgijsbrechts@ucp.pt, https://orcid.org/0000-0002-0846-6855 (JG); robert.boute@vlerick.com, https://orcid.org/0000-0003-2890-2797 (RNB); vanmieghem@kellogg.northwestern.edu, https://orcid.org/0000-0002-8954-7613 (JAVM); denniszhang@wustl.edu, https://orcid.org/0000-0002-4544-775X (DJZ)

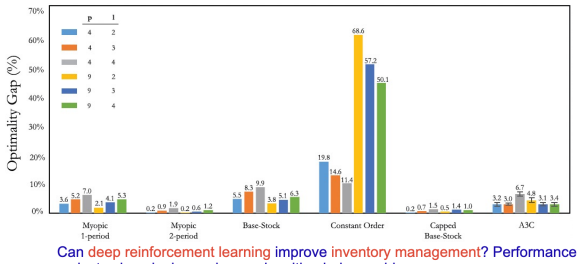
Received: November 5, 2019
Revised: October 7, 2020; July 2, 2021
Accepted: November 11, 2021
Published Online in Articles in Advance: January 17, 2022
https://doi.org/10.1287/msom.2021.1064

Abstract. *Problem definition:* Is deep reinforcement learning (DRL) effective at solving inventory problems? *Academic/practical relevance:* Given that DRL has successfully been applied in computer games and robotics, supply chain researchers and companies are interested in its potential in inventory management. We provide a rigorous performance evaluation of DRL in three classic and intractable inventory problems: lost sales, dual sourcing, and multi-echelon inventory management. *Methodology:* We model each inventory problem as a Markov decision process and apply and tune the Asynchronous Advantage Actor-Critic (A3C) DRL algorithm for a variety of parameter settings. *Results:* We demonstrate that the A3C algorithm can match the performance of the state-of-the-art heuristics and other approximate dynamic programming methods. Although the initial tuning was computationally demanding and time demanding, only small changes to the tuning parameters were needed for the other studied problems. *Managerial implications:* Our study provides evidence that DRL can effectively solve stationary inventory problems. This is especially promising when problem-dependent heuristics are lacking. Yet, generating structural policy insight or designing specialized policies that are (ideally provably) near optimal remains desirable.

Supplemental Material: The online appendix available at https://doi.org/10.1287/msom.2021.1064.

Keywords: inventory theory and control • logistics and transportation • supply chain management • OM-information technology interface

- Some inventory control problems are notoriously challenging: lost-sales, dual-sourcing, multi-echelon, etc.
- A3C algorithm matches the performance of well-designed heuristics, with limited changes to the tuning parameters.
- Generating structural policy insight and specialized near optimal policies remains desirable.



Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems
J Gijsbrechts, RN Boute… - … Management, 2022 - pubsonline.informs.org
… , no topic is hotter today than machine learning and artificial intelligence. We focus on deep reinforcement learning (DRL), the subfield of machine learning that develops "prescriptions" …
☆ 保存 99 引用 被引用次数：291 相关文章 所有 16 个版本

54

54